

04. 贪心法

算法设计与分析

核心思想

- ▶ 贪心策略：每一步都做出当前看起来最优的选择
- ▶ 不回溯：一旦做出选择，不再考虑其他可能性
- ▶ 局部最优 $\rightarrow$ 期望 $\rightarrow$ 全局最优
- ▶ 关键问题：该问题的局部最优选择能否导致全局最优？
- ▶ 适用场景：具有贪心选择性质和最优子结构的问题

贪心法的两个核心性质

【贪心选择性质】

- ▶ 全局最优解可以通过一系列局部最优选择达到
- ▶ 即：存在某个最优解，其第一步就是贪心选择
- ▶

【最优子结构】

- ▶ 原问题的最优解包含子问题的最优解
- ▶ 做出贪心选择后，剩余子问题仍需最优解决

贪心法解题框架

- ▶ 1. 将问题建模为：在一组候选集中做选择
- ▶ 2. 按某种度量标准对候选集排序或组织
- ▶ 3. 每次选择当前度量最优的候选
- ▶ 4. 检查可行性（是否满足约束条件）
- ▶ 5. 将选中的候选加入解集，更新剩余问题
- ▶ 6. 重复直到得到完整解

经典问题：活动选择问题

- ▶ 问题：给定 n 个活动，每个有开始和结束时间，求最多互不冲突的活动数
- ▶ 策略：每次选择结束时间最早的活动
- ▶ 证明思路：设最优解第一个活动为 a_k ，若 a_k 不是结束最早的，用结束更早的替换，不减少活动数
- ▶ 算法步骤：按结束时间排序 $\rightarrow$ 选第一个 $\rightarrow$ 排除冲突的 $\rightarrow$ 重复
- ▶ 时间复杂度： $O(n \log n)$ （排序主导）

经典问题：区间调度 / 区间覆盖

【最大不相交区间数】按右端点排序，贪心选择

【最少点覆盖所有区间】按右端点排序，选择区间右端点

【最少区间覆盖线段】按左端点排序，每次选覆盖起点且延伸最远的

- ▶ 核心思想：排序后，贪心策略通常与端点有关
- ▶ 注意：不同变体的贪心策略不同，需仔细分析

经典问题：哈夫曼编码

- ▶ 问题：给定字符频率，构造最优前缀编码，使总编码长度最短
- ▶ 策略：每次合并频率最低的两个节点，形成新树
- ▶ 数据结构：优先队列（最小堆）维护节点频率
- ▶ 正确性：频率低的字符放深层，频率高的放浅层，总代价最小
- ▶ 时间复杂度： $O(n \log n)$ ， n 为字符种类数

经典问题：最小生成树 (MST)

【Prim 算法】从单一顶点开始，每次选连接树与非树顶点的最小边

→ 适合稠密图，时间 $O(n^2)$ 或 $O(m \log n)$ （堆优化）

【Kruskal 算法】按边权排序，每次选最小边，不形成环则加入

→ 适合稀疏图，时间 $O(m \log m)$ （排序主导）

- ▶ 两者都是贪心：Prim 选最近顶点，Kruskal 选最小边
- ▶ 正确性证明：割性质 (Cut Property)

经典问题：单源最短路径 — Dijkstra

- ▶ 问题：非负权图中，求源点到所有点的最短路径
- ▶ 策略：每次选择距离源点最近的未确定顶点，松弛其邻边
- ▶ 数据结构：优先队列维护当前距离估计
- ▶ 时间复杂度： $O((n+m) \log n)$ （二叉堆）或 $O(m + n \log n)$ （斐波那契堆）
- ▶ 注意：负权边会导致贪心失效，需用 Bellman-Ford 或 SPFA

经典问题：分数背包

- ▶ 与 0/1 背包区别：物品可以分割，取一部分
- ▶ 策略：按单位重量价值 ($v[i]/w[i]$) 降序排序
- ▶ 每次取单位价值最高的物品，能拿全拿，不能拿就拿一部分
- ▶ 时间复杂度： $O(n \log n)$ （排序主导）
- ▶ 对比：0/1 背包不能贪心，必须用动态规划

贪心法正确性证明方法

【交换论证】假设最优解不采用贪心选择，通过交换元素构造新解

→ 证明新解不劣于原最优解，从而贪心选择可纳入最优解

▶

【归纳法】基例成立，假设前 k 步贪心选择正确，证明第 $k+1$ 步也正确

▶

【反证法】假设贪心解不是最优，推出与问题性质矛盾

贪心 vs 动态规划 vs 分治

【贪心】 只做当前最优选择，不回溯，不存储所有状态

→ 快，但仅适用于具有贪心选择性质的问题



【动态规划】 考虑所有子问题，存储中间结果，求全局最优

→ 慢，但适用范围广，保证最优



【分治】 子问题独立，递归求解，不存储重叠子问题

→ 适用于可均分且独立的子问题

贪心法常见陷阱

- ▶ 误区：看到每一步最优就认为是贪心题
- ▶ 反例：0/1 背包用贪心（按价值/重量排序）得不到最优解
- ▶ 反例：找零钱问题，币值 $\{1, 3, 4\}$ ，目标 6，贪心选 $4+1+1=6$ （3枚），最优 $3+3=2$ 枚
- ▶ 判断标准：尝试构造反例，若找不到再尝试证明正确性
- ▶ 建议：先尝试贪心，若无法证明正确性，改用 DP 或搜索